LogiEdge: Edge AI for Cold-Chain Monitoring on FreightBridge Trucks

AIML ZG535 Machine Learning on Edge (EC-I)

Group 15

Student Details:

BITS ID	Name	Contributions
2025ae05381	Disha Shetty H	100%
2025ae05382	Murali Krishna Sai Garisapati	100%
2025ae05420	Rajan Yadav	100%
2025ae05680	Santosh Kumar Behera	100%

Repository: https://github.com/santosh-2025ae05680/logibridge

Introduction

FreightBridge operates 85 refrigerated trucks carrying vaccines and medicines, and has already lost ₹28 lakh of vaccine to a refrigeration failure that went undetected. This report describes LogiEdge, an edge pipeline that classifies the cargo compartment as Normal, Warning or Critical on the truck itself, from temperature, compressor vibration and door events. It alerts without depending on continuous cellular coverage and can be updated and monitored across the fleet. All sensor data is simulated, so the results show that the pipeline works end to end, not how accurate it would be on real trucks. Sections 1 to 5 cover the deployment context and hardware, the sensor pipeline and MQTT design, model deployment, optimisation, and MLOps monitoring. Every figure quoted comes from the project's own scripts and logs in the repository.

1. FreightBridge Deployment Context

We examined latency, bandwidth, connectivity and privacy before choosing an on-truck design. Latency. FreightBridge requires a warning within 90 seconds after a refrigeration fault becomes visible in the sensor data. A failed refrigeration unit can raise temperature by about 1 °C each minute, so waiting for a distant service is risky. A cloud round trip may be quick when coverage is good, but latency alone is not the main problem: network handover, packet loss and unavailable cellular coverage can make the response unbounded. We therefore make the decision locally. The design produces a new six-feature vector every 10 seconds after a 30-second feature window becomes available; the model itself completes an invocation in well under one second on our WSL test machine. From the window size and step, a rough engineering allowance is 30 seconds to collect context, up to 10 seconds to reach the next step, plus processing and alert handling. This is not a measured vehicle fault-to-alert SLA, and we would verify that timing on the selected device before installation.

Bandwidth. The brief asks us to size the system for raw three-axis vibration at 500 Hz, even though our working Python simulator publishes precomputed vibration RMS at 0.5 Hz. Assuming 4-byte floating-point values, raw vibration alone is 500 × 3 × 4 = 6,000 bytes per second, or 518.4 MB per 24 hours using

decimal megabytes. A 1 Hz temperature reading adds 0.3456 MB per day; door events add a small, event-dependent amount. Together the numeric streams are roughly 518.75 MB per truck per day, or ₹51.87 at ₹0.10/MB. For 85 trucks this is about 44.1 GB per day before protocol overhead. Sending only exceptions and short alert records lowers cellular usage considerably; the exact amount depends on the number and size of alerts and should not be presented as a fixed measured saving.

Connectivity. The stated routes lose coverage for 35–90 minutes at seven locations. Cloud-only inference cannot guarantee an alert during those gaps. In our design, MQTT, preprocessing, inference and alert storage continue on the local edge node. When the operations centre becomes reachable, the sync service retries pending alerts and records the acknowledgements. Our live simulation demonstrated two delivered alerts after reconnection.

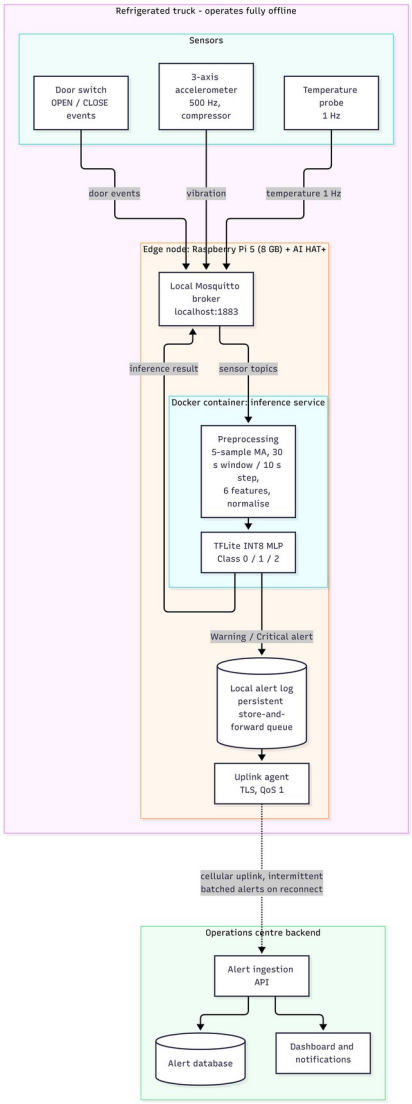
Privacy. Pharmaceutical customers need control over access to cargo-condition records. Keeping raw sensor streams on the truck reduces the amount of sensitive operational data crossing a carrier network. For production, we would add authenticated devices, TLS transport, restricted local file permissions, audit records and defined retention periods. On-device inference reduces exposure but does not by itself prove that all access is authorised.

We compared the Raspberry Pi 5 with AI HAT+ (7.5 W, ₹15,000), Jetson Orin Nano Super (15 W, ₹45,000), and STM32H7-based MCU (0.4 W, ₹3,500). We selected the Pi option because its stated power is inside the 10 W budget, its Linux software can run the Docker/MQTT/Ansible setup, and it costs less than Jetson. The Jetson exceeds the stated budget; the MCU would require a different deployment stack. This is a design decision, not a board-level measurement.

Table 1: Hardware options

Option	Hardware	Power	Price per truck	Within 10 W	Pilot (85)	Full (265)
1	Raspberry Pi 5 + AI HAT+ (Hailo-8L)	7.5 W	₹15,000	Yes	₹12.75 lakh	₹39.75 lakh
2	Jetson Orin Nano Super	15 W	₹45,000	No	₹38.25 lakh	₹119.25 lakh
3	STM32H7 custom MCU	0.4 W	₹3,500	Yes	₹2.98 lakh	₹9.28 lakh

Figure 1. Architecture of the LogiEdge proof-of-concept.



Our design keeps temperature, vibration and door sensors, a Mosquitto broker, preprocessing, TFLite inference and alert storage together on the edge node. In the tested WSL implementation, an HTTP store-and-forward process reads locally saved alerts, retries delivery while the operations centre is unavailable and sends each event after the receiver returns. We observed acknowledgements and database receipts. The test stopped an HTTP receiver container; it did not reproduce a physical cellular outage.

For the hardware exercise, arithmetic intensity is 45 million FLOPs divided by 18 million bytes, or 2.50 FLOP/byte. The Raspberry Pi CPU ridge point is 16 GFLOP/s divided by 12 GB/s, or 1.33 FLOP/byte. Since 2.50 exceeds 1.33, the given workload is compute-bound. Its Roofline lower bound is about 2.81 ms for compute versus 1.50 ms for memory traffic. Reducing arithmetic work is therefore the relevant optimisation. Our small 6–32–16–3 network is not the same 45 MFLOP workload; this exercise estimate should not be confused with measured WSL inference latency.

2. Sensor Pipeline and MQTT Design

Simulator. We used a Python sensor simulator to publish temperature at 1 Hz around the 4 °C setpoint, vibration RMS at 0.5 Hz, and discrete OPEN/CLOSE events. The available modes are none, temp_drift,

vibration and combined. The temperature mode adds a linear drift of 0.08 °C per reading, while the vibration fault changes the RMS distribution toward 1.2 g. A fixed seed makes the generated sequence repeatable, which helped us compare results and investigate failures without changing the sensor inputs between tests. In the final live run, we kept the simulator at normal speed (1×) rather than accelerating time. The combined anomaly ran from second 120 to 240; the full 1,260-second sequence published 1,260 temperature values, 630 vibration values and four door messages. The simulated RMS feed is not the raw 500 Hz signal used in the separate bandwidth exercise.

Our MQTT topics use `logibridge/trucks/{truck_id}/` as the common root. Temperature and vibration RMS are published with QoS 0, since the next reading quickly follows. Door, status and alert messages use QoS 1 to avoid losing important state changes; inference outputs use QoS 0 because each 10-second result supersedes the previous one. We keep status retained for late subscribers. Within the test, Mosquitto is isolated on `logibridge-net`; the HTTP alert synchronizer is separate from MQTT and uses local JSONL plus an acknowledgement journal. We do not claim that MQTT QoS alone guarantees delivery across a broken cellular link.

Figure 2. MQTT topic tree.

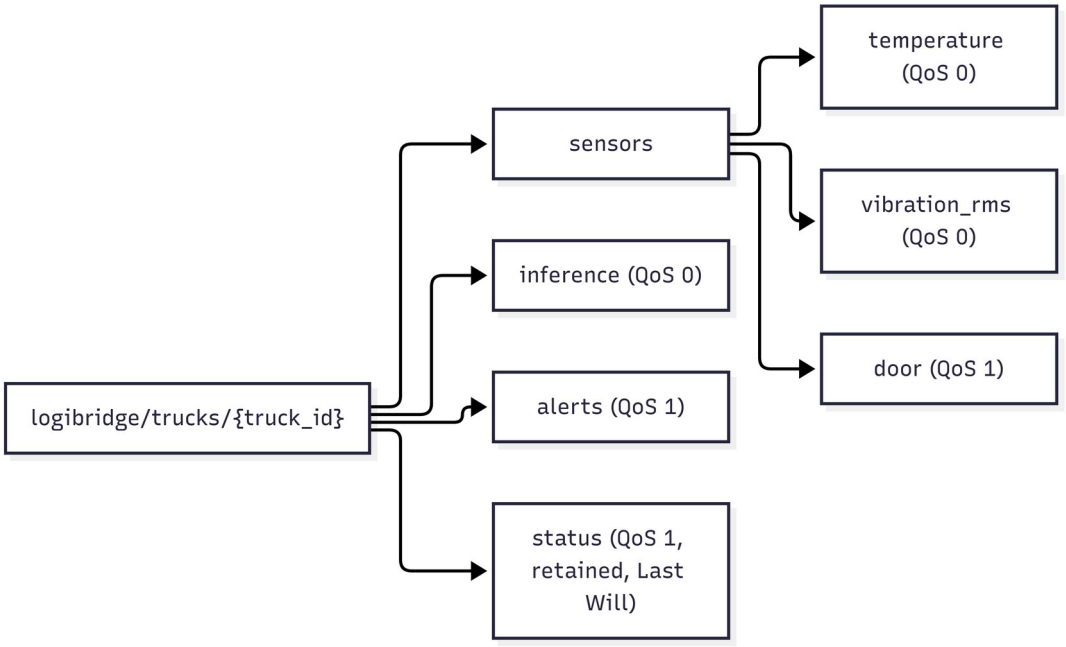


Table 2: QoS per topic

Topic	Rate	QoS	Retained
sensors/temperature	1 Hz	0	no
sensors/vibration_rms	0.5 Hz	0	no
sensors/door	on event	1	no
inference	every 10 s	0	no
alerts	on Warning or Critical	1	no
status/{client}	on change	1	yes (Last Will)

Our temperature and vibration streams first pass through separate five-sample moving averages. We then form 30-second sliding windows at 10-second steps, rather than waiting for disjoint blocks; the overlap makes it easier to observe an event beginning near a window boundary. Each window supplies six features: temperature mean, standard deviation and rate of change in °C per minute, plus vibration

RMS, peak and kurtosis. We apply the saved training mean and standard deviation to each feature and clip extreme normalised values to ±10. Runtime processing loads `training_stats.npy` rather than recalculating statistics from abnormal live traffic. This matters because adapting the baseline during a failure could make the very fault we need to detect appear normal. We checked the normal, temperature-only, vibration-only and combined feature outputs separately before testing the MQTT pipeline.

We evaluated the required shifted-normalisation experiment on the same 57 synthetic validation windows. With ±10 clipping, Keras FP32 and INT8 TFLite stayed at 100% accuracy using correct statistics or statistics shifted by +3σ or −3σ. Without clipping, the FP32 Keras experiment reported 40.35% even with the unshifted statistics, while INT8 stayed at 100%. This shows that our preprocessing and quantisation range are closely coupled; it does not show that statistics shifts are harmless in real sensor data. The CSV evidence is `training/results/stats_shift_results.csv`.

Table 3: Validation accuracy (57 windows) under shifted statistics

Model	Clip	Correct statistics	+3σ	-3σ
FP32 Keras	±10	100.0%	100.0%	100.0%
FP32 Keras	off	40.4%	40.4%	40.4%
INT8 TFLite	±10	100.0%	100.0%	100.0%
INT8 TFLite	off	100.0%	100.0%	100.0%

Data fusion. We chose feature-level fusion: the temperature and vibration processing steps each produce their own measurements, and we concatenate those measurements into one six-value input before classification. This preserves information from both sensors while avoiding the work of synchronising every raw sample across streams with different rates. Data-level fusion would require resampling or padding the high-rate waveform and lower-rate temperature values to a common timeline. That adds traffic and memory without a clear benefit for our small MLP. Decision-level fusion would require separate classifiers and a rule to combine their outputs, increasing deployment and calibration work. Door events remain a separate operational signal in the demonstration rather than being silently counted as a seventh model feature. This separation keeps the six-feature architecture reproducible and makes it clear which evidence actually drives the three-class prediction.

3. Model Deployment and Pipeline Mapping

Table 4. Our ten-stage Edge ML pipeline, mapped to the project.

#	Stage	What happens in LogiEdge
1	Problem definition and constraints	Three classes, a 90 s alert SLA, a 10 W power budget and fully offline operation set the requirements.
2	Data acquisition	A simulator publishes temperature, vibration RMS and door events to the local Mosquitto broker with per-topic QoS.
3	Preprocessing	A 5-sample moving average, 30 s windows with a 10 s step, and z-scoring with frozen training_stats.npy and a clip at ±10.
4	Feature engineering and fusion	Six features (temperature mean, standard deviation, rate; vibration RMS, peak, kurtosis) fused at feature level.
5	Model training	A 6-32-16-3 MLP (803 parameters) trained on 285 windows from three simulator modes, with a time-ordered split and a two-window purge.
6	Evaluation and validation	100% accuracy on 57 held-out windows (gate 88%) and 100% Class 2

#	Stage	What happens in LogiEdge
		recall (gate 95%), plus the 3σ shifted-statistics experiment.
7	Optimisation	INT8 post-training quantisation (M2, 3.38 KB) and 35% neuron pruning with INT8 (M3, 2.94 KB), compared by benchmark and Pareto analysis.
8	Conversion and packaging	TFLite models inside a python:3.11-slim container with the model as the last layer and a MODEL_PATH override.
9	Deployment and OTA update	A local registry with versioned tags and a seven-task Ansible playbook; a canary rollout is recommended.
10	Monitoring and maintenance	PSI on P(Normal) over the last 100 inferences every 60 s, alerting above 0.25, with a reference tied to the deployed model.

We generated 285 usable labelled windows across Normal, Warning and Critical, with 228 used for training and 57 held out for validation. We trained a 6–32–16–3 multilayer perceptron with 803 parameters, then exported three deployment candidates. The held-out synthetic set contains 23 Normal, 17 Warning and 17 Critical windows. Each model classified these 57 examples correctly in our reported experiment, including all 17 Critical windows. The high accuracy is useful evidence that conversion preserved the decisions on this dataset, but we would not treat it as proof of generalisation to different trucks, refrigeration systems or environmental conditions. Our final choice therefore also considers resource usage, monitoring and a staged deployment plan, not accuracy alone.

Our M2 baseline uses image tag `1.0`; we rebuilt image `1.1` with the M3 model and compared actual registry layers. Most layers were reused. The measured compressed image size in the final run was 119.649 MB per truck; the layers missing from the baseline totalled 0.00573 MB per truck. For 85 trucks the calculation is 10,170.18 MB for full pulls against 0.4875 MB with cache reuse, or 99.9952% less transfer. This assumes that the older layers remain cached on every vehicle and does not represent measured cellular traffic.

Table 5: Bandwidth for 85 trucks

Update method	Per truck	85 trucks	Cost at ₹0.10/MB
Full image pull	119.649 MB	10,170.18 MB	₹1,017.02
Layer-cache OTA	0.00573 MB	0.4875 MB	₹0.0487

The assignment assumes a 280 KB model replacement every six weeks across 85 trucks at ₹0.10/MB. Under that separate business assumption, full replacement transfers 23.80 MB and costs ₹2.380 per cycle; a successful ten-truck canary followed by the remaining 75 trucks transfers 23.92 MB, including small health telemetry, and costs ₹2.392. On-device shadow comparison with daily summary uploads is estimated at ₹2.440. These are model-update transfer assumptions, distinct from our measured Docker image-layer sizes. We prefer canary for the actual fleet because a fault can spoil pharmaceutical cargo. Ten trucks can be monitored before extending the change to the other 75, and the retained previous model supports a local rollback during unreliable connectivity. This policy is proposed in the report; the single-node Docker/Ansible demonstration is not a completed 85-truck rollout.

Table 6: Cost per update cycle (280 KB model)

Strategy	MB per cycle	₹ per cycle	₹ per year	Trucks exposed
Full replacement	23.80	2.380	20.7	85
Canary (10, then 75)	23.92	2.392	20.8	10
Shadow, on-device comparison	24.39	2.440	21.2	0

Strategy	MB per cycle	₹ per cycle	₹ per year	Trucks exposed
Shadow, upload all outputs	537.88	53.788	467.4	0

4. Optimisation and Pareto Analysis

We compared three TFLite exports: M1 FP32, M2 full INT8 post-training quantisation, and M3 structured-neuron pruning followed by INT8. The original MLP has 803 parameters, while M3 uses a smaller 6–21–10–3 topology with approximately 400 parameters. M3 produced the smallest file (2.94 KB) and tensor memory requirement (545 bytes). Our recommendation is based on these measured size advantages and the synthetic validation outcome, rather than a claim of real-truck reliability.

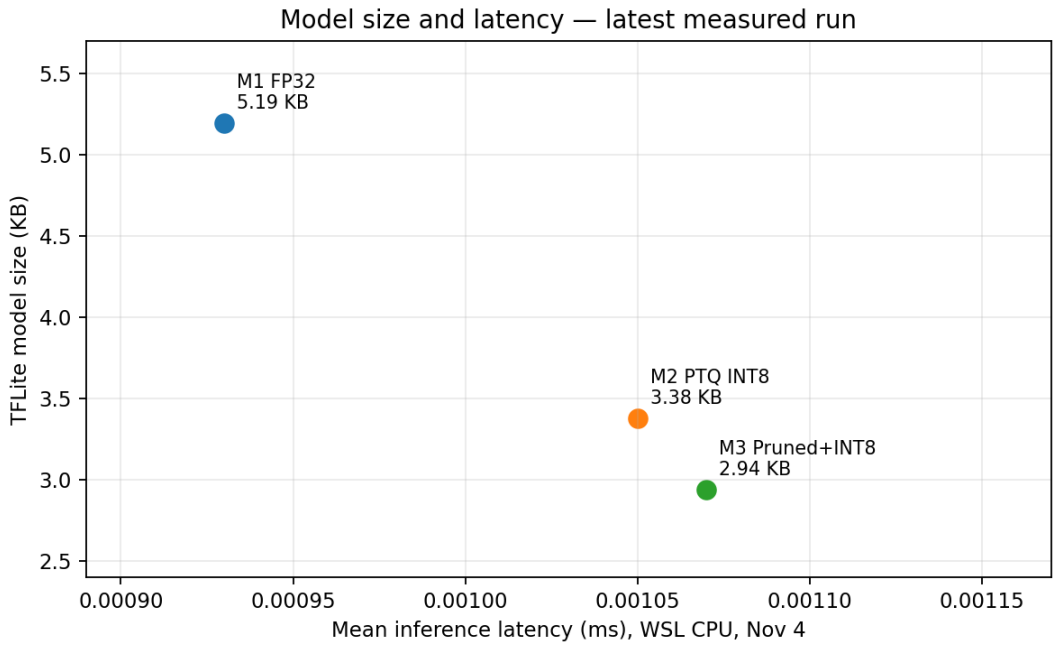
For each model, we ran five trials, with ten warm-up calls and 200 timed inference calls per trial. We used our 16-logical-core WSL laptop, not a Raspberry Pi. Energy per inference is estimated using a 28 W TDP and a 1.75 W share for one busy logical core. The differences between models are very small and can change when the benchmark is repeated, so we do not claim a reliable hardware speedup from this laptop test.

Table 7: Benchmark results

Variant	Mean latency (ms)	p95 latency (ms)	File size (KB)	Accuracy (%)	Energy (mJ)
M1 FP32	0.00093	0.00109	5.19	100.00	0.001626
M2 PTQ INT8	0.00105	0.00112	3.38	100.00	0.001835
M3 Pruned+INT8	0.00107	0.00108	2.94	100.00	0.001876

In the 4 November benchmark, M1 had the lowest measured mean inference latency at 0.00093 ms, while M3 had the smallest model file at 2.94 KB and the lowest P95 latency at 0.00108 ms. M2 remained close on latency. The Pareto chart identified both M1 and M3 as non-dominated on the selected measurements, so our recommendation is not that M3 wins every benchmark. We favour M3 for its smaller model and tensor memory while keeping 100% Critical recall on the synthetic held-out set. The differences between mean latencies are tiny on this CPU and should not be interpreted as robust rankings across hardware or operating temperatures. We would repeat the measurement on the real Raspberry Pi, check tail latency under competing workloads, and validate warning and critical failures using independently collected sensor traces before approving the model for shipment.

Figure 3. Latest WSL CPU model-size and latency comparison (4 November 2026).



Observed trade-off: M1 is fastest; M3 is smallest. Both appear on the size/mean-latency frontier.

We recommend deploying M3 to the simulated fleet pilot only after a cautious staged acceptance process. It meets the exercise's greater-than-95% Critical recall threshold on the held-out synthetic set, and its 545-byte tensor allocation is below M1 and M2. The 90-second SLA is mainly a windowing and alert-transport concern; individual model calls are much faster than the ten-second inference cadence on our host. The Pi 5 choice is well above these model-memory needs, but actual flash, SRAM, latency, power, cellular reliability and out-of-distribution recall must be tested on hardware before production use.

5. MLOps Monitoring and Reflection

We built the M3 reference distribution from 300 clean Normal windows, with four bins of the probability assigned to Normal (`p_normal`). During our latest live test on 4 November, the simulator ran at normal 1× speed for 1,260 seconds. We introduced the combined fault from second 120 to 240. The monitor evaluated PSI every 60 real seconds using up to the latest 100 predictions. PSI started at 0.000 (`n=8`), rose to 2.369 at the first alert, peaked at 4.604, and eventually fell to 0.000 with a complete 100-prediction window. This shows recovery as old fault-related predictions leave the window. The experiment measures an output distribution change on synthetic sensor data; it is not a physical truck or real cellular test.

Table 8. Actual PSI transition during our real-time run on 4 November 2026.

Phase	Time	PSI	Status
Baseline	16:01:50	0.000	ok; n=8
First alert	16:02:50	2.369	ALERT; n=14
Peak	16:03:50	4.604	ALERT; n=20
Declining	16:14:50	0.932	ALERT; n=86
Near recovery	16:19:50	0.313	ALERT; n=100

Phase	Time	PSI	Status
Recovered	16:20:50	0.000	ok; n=100
Delivered alerts	after receiver restart	2	ACK and stored

We introduced a change in the input stream by increasing temperature and vibration during the combined fault, while leaving model weights unchanged. The monitor watched the probability assigned to the Normal class, `p_normal`, not just the highest class probability. This matters because a confident Critical prediction can still have a high maximum probability; monitoring maximum confidence alone could hide a switch from Normal to Critical. We used four bins and a 300-window Normal reference, then calculated PSI every 60 real seconds using at most the latest 100 predictions. The initial value was 0.000, the peak reached 4.604, and after the abnormal predictions had moved out of the sliding buffer PSI returned to 0.000 with `n=100`. These results show a shift in model-output distribution and recovery under our controlled simulation; PSI alone cannot prove the physical cause of a refrigeration failure.

Our seven-task Ansible playbook installed M3 and its matching reference on the local WSL node. The first run reported `ok=7 changed=3`; a second run with no intervening changes reported `ok=6 changed=0 skipped=1`. Task four intentionally stops and removes an old container so the replacement can use the same name. The unchanged second run leaves the service running. This establishes idempotency on localhost, not deployment to 85 real trucks.

One practical difficulty was preserving the monitor's last 100 predictions while still showing a complete recovery in a real-time test. Our first 320-second run detected drift, but ended before enough clean predictions had replaced abnormal ones. We initially used a faster demonstration, then changed to real-time sensor generation and 60-second PSI checks to match the stated monitoring requirement. The final schedule ran for 1,260 seconds at 1×, with the fault present from second 120 to 240, and reached both alert and recovery states. Another issue was making the OTA deployment repeatable. Stopping and removing an old inference container allowed a fresh model and reference to be mounted under the same service name, while a second Ansible run made no changes. These experiences changed how we planned our tests: we now keep a measured baseline, a sustained fault, enough recovery time and a separately verifiable deployment record.

Before attempting fleet rollout, we would isolate the MQTT and HTTP components with mutual authentication and TLS, add a hardware watchdog and reliable cellular-link health checks, and verify the store-and-forward log across a real device reboot. Our current drift monitor already includes a stale-inference check, and the sync process already persists acknowledgements. These areas need field testing, not a claim that they are missing from the implementation.

Conclusion

Our group demonstrated MQTT sensor streaming, 30-second feature windows, TFLite classification, locally stored alerts, delivery after operations-centre reconnection, a small model-only OTA update, repeatable Ansible deployment and PSI drift monitoring. In the 4 November run, PSI rose from 0.000 to 4.604 and returned to 0.000 with 100 recent predictions. Two Critical alerts were received after the service came back online. M3 remained the smallest model and met the synthetic Critical-recall requirement. Before any real fleet deployment we would need physical Pi 5 measurements, recorded truck data, secure communication and tests across genuine cellular outages.